视频透视自适应校正系统

摘要

书本、纸张等平面目标在倾斜或旋转拍摄时会产生透视畸变，版面歪斜、文字拉伸，不能直接当作正视画面来阅读、归档或送入识别算法。而现有“拍照后逐张处理”的方式，在背景杂乱或目标移动时仍需人工框选四角，难以适应视频拍摄的场景。

本作品构建了一套**基于PYNQ**平台的视频透视自适应校正系统解决此问题。系统以目标的物理外边缘作为定位依据，**把传统视觉和AI分成两个阶段配合使用**：常规情况下先由传统算法检测轮廓、拟合四条边，并校验它们之间的几何关系。只有在常规搜索找不到有效目标时：比如目标边缘不清晰、颜色与环境接近，或边缘有轻微遮挡、反光，才调用轻量化分割AI模型给出候选区域和初始四角。候选四角通过当前帧验证之后，交由跟踪模块接管，**此后不再逐帧运行神经网络**。跟踪模块用**双向光流**复测跟随目标运动，只在边缘证据不可靠或运动突增时才重新求解**单应变换**。

在软硬件分工上：控制、定位和跟踪放在处理系统PS侧运行，像素重映射、锐化和分段线性光补则由可编程逻辑PL侧承担，PS与PL之间以异步任务解耦，并通过帧序号校验保证帧数据一致。系统还通过相机复用、帧序号校验、异步任务和单一显示提交者这几种机制，减少了模式切换时的重复初始化，也避免了多路输出造成的画面不一致。界面上提供自动定位、AI校正、手动四角、分辨率选择和调节参数（如锐化、光照补偿）等操作。当定位短暂失效时，系统会在限定时间之内继续用最后可信的四角去处理当前相机画面，画面保持连续更新。整机由开发板、触摸屏和USB摄像头组成，采集、定位、解算、重映射和显示全部在板上完成。

系统已在PYNQ-Z2上完成部署并通过板端实测，**累计运行数十小时、经历千余次交互会话未出现崩溃**，能够持续稳定地完成实时校正与显示。

关键词：透视校正；FPGA；软硬协同；单应变换；光流跟踪；轻量语义分割。

目录

摘要.....1
第一部分　作品概述.....3
1.1 功能与特性.....3
1.2 应用领域.....3
1.3 主要技术特点.....3
1.4 主要性能指标.....4
1.5 主要创新点.....4
1.6 设计流程.....5
第二部分　系统组成及功能说明.....6
2.1 整体介绍.....6
2.2 硬件系统介绍.....7
2.3 软件系统介绍.....10
第三部分　完成情况及性能参数.....15
3.1 整体介绍与工程成果.....15
3.2 特性成果.....16
3.3 训练数据与模型验证.....19
3.4 实时运行与验证边界.....20
3.5 PL 校正核的设计迭代与验证.....21
第四部分　大模型协作与技能包.....23
4.1 智能体工作流.....23
4.2 提示词与自我纠错.....24
4.3 技能包的提炼过程与边界.....26
4.4 AMD ROSS 智能体套件的使用心得与优化建议.....26
第五部分　复现说明.....28
第六部分　总结.....29
6.1 可扩展之处.....29
6.2 设计总结.....29
第七部分　参考文献.....31

第一部分　作品概述

1.1 功能与特性

系统自动识别画面中近似平面的目标，定位其物理外边缘的四个角点，把倾斜、旋转或平移拍摄到的画面实时校正成端正、便于阅读的矩形，并且可以在"原图与校正图并排"和"只显示校正图"两种方式之间随时切换。默认情况下自动算法优先工作，遇到困难目标时由AI补充定位，用户也可以自己手动指定四个角点。输入支持480P和600P两个档位，另外还提供放大阅读、截图和相册功能。锐化和光补两个滑条拖动后立即反馈，参数统一在下一帧开始时生效。实时界面上同时显示当前校正结果的来源（AI或传统算法）和输出帧率，方便判断这一帧是由哪一种方式定位的。目标运动时由光流跟踪逐帧跟随，省去逐帧重新搜索。整套系统在开发板上独立运行，相机从USB口进、校正后的画面从HDMI口出，交互靠触摸屏，处理全程都在板上完成。

1.2 应用领域

透视校正现有的做法大致有三类：手机扫描类应用需要逐张手动框选四角；专用文档相机固定在支架上，靠机械结构保证正对；遥感和摄影测量里的几何校正依赖标定或已知控制点。三者都不处理“相机自由摆放、目标随时移动”的场景。本系统把定位、跟踪和像素级重映射串成一条实时链路，四角由算法自动给出，用户只在必要时介入。

本作品适用于桌面上的书本阅读、纸质资料展示、辅助阅读和嵌入式视觉教学；也可以用来跟随平板的外形或者呈现屏幕内容；还可以用在传送带上——相机斜装在一侧，对快速通过的快递面单这类平面目标持续跟踪四角，边移动边输出正面画面，供后续读码或识别使用。系统处理的对象是摄像机视野里的近似平面物体；对于平板这类设备，标注和定位覆盖的是包含黑色边框在内的机身物理外缘。边角轻微遮挡或轻微反光不影响定位，分割模型在训练时就覆盖了这类情况，仍然可以得到四角。至于书页弯曲、被完全挡住因而看不见的内容，以及强反光淹没的区域，这些不是单应性变换能够恢复的，系统不对此做保证。

1.3 主要技术特点

整条链路按"候选定位、当前帧验证、持续跟踪"三个阶段分工。传统轮廓与边缘方法负责快速定位，AI分割在目标难以提取时给出空间先验。两者给出的四角先过几

何约束，再由当前帧边缘证据确认，之后交给跟踪策略逐帧更新位置。PS侧做事件调度，并解算几何参数；PL侧承担大量重复的像素处理。神经网络不进入逐帧校正的主循环，CPU占用因此降下来。异步结果都带帧序号、请求标识和页面代次，旧任务的结果不会串到新场景上。跟踪以双向光流复测为主，只在必要时才重新求解单应矩阵。PS与PL同在一颗Zynq器件上，整条链路的输入输出都在板上闭环，不依赖主机。

1.4 主要性能指标

表1　交付版本的主要性能指标

指标	数值
四角定位耗时	自动检测0.38s（中位）；AI候选返回2.49s（均值）
单帧跟踪耗时（全部帧）	自动17.7ms；AI辅助16.7ms
光流复测	单次事务中位71.6ms
静态保持单帧	35.3～36.6ms（480P静止）
PL单帧处理	自动10.7ms；AI辅助10.1ms
平均帧率（单幅校正，静动态合计）	自动23.2fps；AI辅助21.1fps
AI角点误差	4.6px
本地AI部署	MobileNetV3-Small + U-Net，约137万参数；输入256×192RGB；NCNN FP32
校正核吞吐	4.686个时钟周期/像素（初版11.99，提速2.56×）
PL资源	LUT33.02%、FF22.68%、BRAM86.43%、DSP20.91%
实现时序	WNS +0.650ns、WHS +0.015ns

1.5 主要创新点

（1）AI与传统算法分工：传统轮廓负责快速定位，AI只在困难场景补充区域先验；候选四角须经当前帧边缘证据验证后才交给光流复测，网络不进入逐帧主循环。

（2）PS与PL的分工与流水：PS负责定位、跟踪与透视求解，PL用全流式三级流水线做重采样与增强，两侧在同一时刻处理的是不同的帧——PS解算下一帧的几何时，PL正在重映射当前帧；同一比特流支持480P与600P两档。

（3）光流复测按需触发：只有边缘证据不可靠、运动突增、漂移累积或锚点帧龄到期时才做一次双向LK，触发后还要过四判据门控——四边齐全、最小支持率不低于

0.60、最大拟合残差不大于1像素、修正量不超过搜索半径的0.65倍；锚点到期只做一次局部复验，把搜索范围压到最小。

（4）物理外边缘统一口径：数据标注、模型输出与跟踪目标同取物理外边缘，平板含边框。

（5）可证明的实时性：每帧提交前用定点区间算术证明行缓存访问不越界，不通过则自动分块或拒帧。

（6）宽度换深度与整数域递推：512宽省下的BRAM换384行深度，分块由4降到1；坐标递推在精确整数域进行，每像素仅3次加法。

1.6 设计流程

开发按“三条线并行、两次交汇”的方式组织：三条线各自推进，在两个节点上汇合。



图1　三条研发线的并行推进与两次交汇

主线承担算法与硬件，分五段推进。定型解决“做什么”：在候选方案中做五维对比，确定平面自适应校正这条路线，形成定稿。打通解决“能不能跑”：从零建成PL校正核，覆盖块设计、流式重映射与坐标发生器，先让整条链路在板上运行起来。求对解决“算得对不对”：加入帧身份校验与色彩通路，把找四角从PC端搬到板上，以实拍样本判废率归零为通过标准。求快解决“连不连续”：把取帧、解算、重映射、显示四段做成重叠流水，并降低显示合成的开销。攻坚解决“够不够快”：对校正核逐轮压缩每像素占用的时钟周期数，同时缩减600P预览开销，最后完成交付打包。

第二条线承担交互与跟踪，在主线打通之后加入，分三段推进：先把触摸操作做成闭环，再让系统在静止与跟丢两种状态下都能自行恢复，最后把同帧成对显示和双

档位切换做到不断流。这条线关心的核心是“在人手里用起来连不连续”。

第三条线是模型，在收口阶段加入，分两段：先补训练数据，再把轻量分割网络接进定位环节——只提供初始四角，拿到之后立即交回传统跟踪，不进入逐帧通路。

三条线之间有两次交汇。第一次，主线并入第二条线的代码库，此后“够不够快”与“连不连续”在同一份代码上同时推进；第二次，三条线合成同一个版本号，交付版本从这时起才正式存在。

全程的放行条件是同一条：先在测试平台上逐字节通过，再上板实测；任一条不满足就停在当前段，补齐后再继续。

第二部分　系统组成及功能说明

2.1 整体介绍

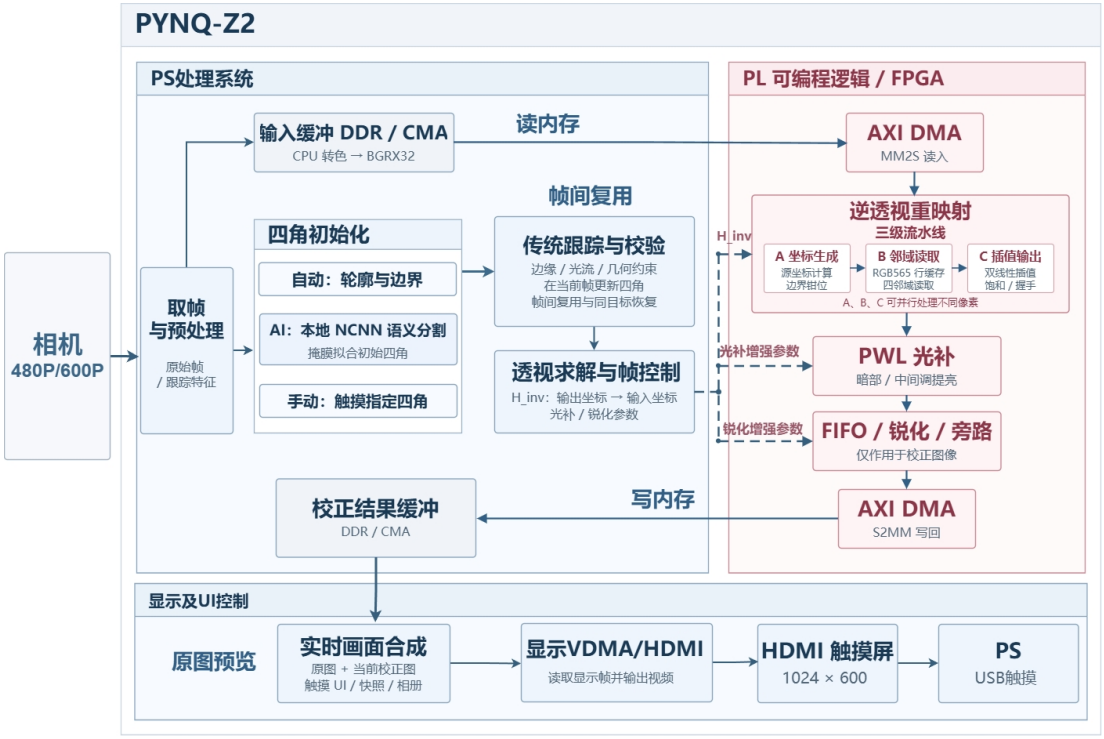


图2　软硬件协作框图

相机按480P或600P采集，原始帧经取帧与预处理后由CPU转色成BGRX32，写入DDR中的输入缓冲。四角初始化有三条路径：自动定位走轮廓与边界，AI辅助调用本地NCNN分割网络、用掩膜拟合出初始四角，手动则由触摸指定。三者都汇入传统跟踪与校验，由四边证据、双向光流与几何约束在当前帧更新四角，并支持帧间复用与同目标恢复。跟踪结果交给透视求解与帧控制，算出把输出坐标映射回输入坐标的

H_inv，连同光补与锐化参数经AXI4-Lite下发到PL。

PL侧由AXI DMA从DDR读入像素流，依次经过三级流水的逆透视重映射、PWL光补与锐化旁路，再由第二个DMA写回DDR中的校正结果缓冲。

显示环节把原图与当前校正图合成，经显示VDMA送到1024×600的触摸屏；触摸事件从USB回到PS，只改变状态和参数，不另起第二个相机读取者或显示写入者。

2.2 硬件系统介绍

系统以PYNQ-Z2开发板为核心，板载XC7Z020（Zynq-7020，约5.3万个LUT、140块BRAM），PS为双核ARM Cortex-A9，与PL通过片上AXI互联。外设是一只USB2.0相机和一块1024×600的HDMI触摸屏，相机提供480P与600P两档，触摸经USB回连PS。PL主时钟83.333MHz，对外接口契约为remap1.5，输入格式BGRX32；配套的bit与HWH文件描述了PL侧的外设、地址映射与接口。从YUYV到显示或处理所需格式的转换由PS侧的原生代码完成，PL只负责像素重映射与画质增强。

2.2.1 PL 校正核与行缓存架构

校正核按照A、B、C三级流水线组织。A级用"行首初值加行内递推"的方式生成源坐标并完成边界钳位，其中多路并行除法器保证了这一级的吞吐。B级根据坐标FIFO给出的地址，在两个存储体上并行读出行缓存里的四个邻域像素；C级完成双线性插值并把结果饱和输出。三级各自占用约2到4个时钟周期，稳态下整核的吞吐为每像素4.686个时钟周期。



图3　校正核的ABC三级流水线结构（稳态吞吐4.686个时钟周期/像素）

我们把校正核分成三级，是因为这几步之间没有真正的先后依赖。取坐标、边界钳位、读邻域、插值和输出如果串在一台状态机里，每个像素要花十来个时钟周期；其中行缓存读口只占四个时钟周期，其余时间都在等彼此独立的步骤完成。拆成并行的三级之后，稳态吞吐由最慢的一级决定。

这里的"三级"指功能级的划分，每一级内部还有更细的状态层次。A级包含四个状态，坐标发生器内部是十六路并行除法单元加一个结果先进先出队列，输出比请求晚两个时钟周期到达。B级用两个时钟周期读完四个邻域像素：偶、奇两个存储体并行工作，同一行左右两列在一个时钟内读出，两行因此只需两个时钟周期。C级三个状态，其中等待状态占一个时钟周期，正好对上插值单元内部的两级寄存器延迟。插值单元必须拆成横向与纵向两级：合并到一个时钟周期内时，综合报告给出的最差路径是16.9ns，在10ns的周期下无法收敛。

A级负责坐标生成，是整个核里最需要仔细处理的部分。如果每个像素都重新计算一次单应变换的六个乘法和三次加法，吞吐会被乘法器链拖住。因此采用"行首初值加行内递推"：每行行首算一次初值，之后向右每走一个像素，三个累加器各自只增加一个常量。这里有一个容易踩的陷阱：若把Q16.16的增量直接累加，每一步都会丢掉半个最低位，走完1280个像素后误差累积到640个最低位，相当于白白损失0.01像素精度。本设计改为在未移位的精确整数域里累加，只在最后输出时移位一次。

我们在坐标除法上也做过权衡。齐次坐标归一化本质上是一个逐像素除法。若顺序执行，一个坐标要花掉六十个时钟周期。本设计改为N路并行，各除法器按轮转顺序依次启动。由于延迟完全相同，完成顺序天然等于启动顺序，结果可以直接按序写入FIFO，不需要额外的重排电路。这里的前提是除数必须在启动时锁存：并行度越高，输入总线换得越快，若除数直接取自活输入，同一组操作数算出的商就会偏。因此并行化之后，我们在不同并行度之间做逐字节比对，把"用了同一个子模块所以结果一样"这类推理排除在外。

B级从行缓存中取出双线性插值所需的四个邻域像素。行缓存组织成环形的若干行，地址由"行号对缓存行数取模"再乘以有效行宽得到；为了在一个时钟内读出左右相邻两列，缓存按列地址最低位分成偶、奇两个体，各有一个独立读口。总容量仍然是行数乘以有效行宽，没有多占一块BRAM，只是把读带宽翻了一倍。当缓存行数不是二的整数次幂时，不能直接写取模表达式让综合器生成通用取模电路，那样会在地址路径上引入组合逻辑，本设计改用预计算加条件减法完成回绕。

C级完成双线性插值。三个颜色通道走并行的数据通路：分时复用同一套乘法器虽然能省DSP，但会把每像素延迟从两个时钟周期拉长到六个，对吞吐的伤害远大于资源上的收益。插值结果经饱和处理后输出，并在最后一个像素上给出帧结束标志，与

行缓存的读写地址共同构成流控握手。

参数通过一组AXI-Lite寄存器下发，地址空间0x00到0x30。0x00控制，0x04状态，0x08与0x0C写入输入和输出尺寸，0x10到0x2C共八个寄存器存放Q16.16的单应系数，0x30是只读版本寄存器。软件每帧下发参数后会把十个寄存器全部读回比对，不一致立即报错。版本寄存器让软件在启动时确认，板上加载的比特流是否与预期一致。

开发过程中我们逐项排除了大部分设计空间，校正核最后只留下一条固定的数据通路。表2列出了当时认真考虑过、最终没有采用的四个方案及其实测代价。其中最直观的是整帧缓冲一项：完成同样的重映射，核内行缓存方案耗时21.9ms，而需要读写整帧的参考实现为68.9ms，多出来的时间几乎全部花在搬运上。核的"简单"是逐项量化之后收敛出来的结果。

表2　被排除的设计方案与实测代价

采用方案	代价	最终方案与实测
整帧缓冲代替行缓存	640×480的YUYV帧约600KB，超过整片BRAM的余量，只能落在DDR	改为O(行)行缓存：核内21.9ms，落DDR的参考实现68.9ms
每像素重算单应坐标	每像素6次乘法+ 3次加法，约60个时钟周期	行首初值+行内递推，坐标生成降到3次加法
坐标除法顺序执行	每个坐标约49个时钟周期	16路并行、轮转启动，降到3.06个时钟周期/坐标
行缓存分4体加宽读口	BRAM96 → 128块（91%），布局布线不通过；吞吐仅7.160 → 7.162	按列低位分2体：容量不变，四邻域2个时钟周期读完

除了性能，校正核还承担一项容易被忽略的责任：行缓存的访问必须是安全的。环形行缓存允许写指针回头覆盖最老的一行，前提是读侧永远不读比缓存深度更老的数据。一旦目标倾斜角变大，一行内需要的源行跨度可能超过缓存深度，读侧就会拿到被覆盖的内容，画面表现为无法解释的色块。本设计把安全性做成可证明的：每帧提交之前，用定点区间算术沿所有输出像素把源坐标的取值范围推一遍，证明最深的一次访问仍在缓存深度之内。不通过时自动把该帧拆成更小的分块，拆分后仍不通过则拒绝这一帧。

从系统约束看，PL侧的结构做到这一步已经够用。USB2.0实际可用带宽约35MB/s，640×480的YUYV帧在30fps下已占去18.4MB/s。继续提高分辨率或帧率，都会被这条链路卡住。校正核的处理能力是17.8兆像素每秒，相机只能提供9.2兆像素每

秒，余量接近两倍。当前设计的BRAM也已用掉86.4%，没有余量再开第二条并行通路。所以更合理的方向是把这一条通路做到底。

行缓存按实际输入宽度512像素、深度384行分配，格式为RGB565。裁剪后的实际宽度不超过328像素，若仍按1024的上限分配，约37.5%的BRAM会长期闲置。我们把节省下来的宽度让给深度，整帧的源行跨度可以一次装下，分块数由4降到1；每个分块的证明、参数下发和DMA启动开销也随之消除。

2.2.2 图像增强与输出链路

校正核之后是图像增强。两级都走AXI-Stream，按像素流处理，只保留行级缓存。光补采用分段线性的暗部提升，因此不需要为局部直方图分配大块缓存，硬件上占0BRAM、1DSP。锐化由现有的PL模块实现，效果受单块输出宽度与拼接边界策略约束。目前的亮度映射是全局的，还没有做到空间局部自适应。

2.3 软件系统介绍

2.3.1 软件组织与数据流

板端软件为单进程多线程结构，按职责分为五层：会话与采集、传统检测、AI服务、混合调度与跟踪、硬件与显示，各层的输入输出见表2。采集线程负责取帧并打上帧序号与页面代次；检测与AI给出候选四角，调度层完成验证与跟踪；硬件后端把透视参数下发给PL，收回校正后的图像；显示线程是唯一的HDMI提交者，按当前模式把原图与校正图合成输出。AI服务常驻独立线程，只在需要时被唤醒，推理结果带请求回执，过期结果直接丢弃。

表3　软件模块及其主要输入输出

模块	主要输入	主要输出
采集与会话	相机档位、页面事件	图像、帧序号、页面代次
传统检测	当前图像、点选位置	候选四角、边缘及几何证据
AI服务	请求图像、目标选择上下文	分割掩膜、候选四角、请求回执
混合调度与跟踪	候选、当前帧、历史可信四角	跟踪四角、来源、有效状态
硬件与显示	映射参数、当前像素、增强参数	校正图、原图、FPS与进度

2.3.2 传统自动定位

自动路径利用亮度、色彩和边缘信息产生轮廓候选，再把这些近似四边形进一步

精化为目标的物理外缘。轮廓拟合本身只是提出候选，之后还要检查凸性、面积、边长关系以及当前帧上的四条边证据，防止把背景纹理或者书页里印刷的矩形当成外框。与单纯挑选最大掩膜的做法相比，轮廓候选能够提供多个目标之间的边界分离信息。[1]

自动搜索的设计目标是尽量优先拿到低延迟的定位机会。在当前的恢复控制逻辑里，完全没有候选时的搜索时间是1.5秒，已有候选的情况下总验证时间上限为2.8秒；只有超过了这个阶段预算，才会进入带有请求次数限制的AI补充定位。上面这些数值是控制条件，每次切换的实际总时长还要加上任务调度、图像准备、推理和当前帧验证等环节。

2.3.3 AI与传统算法的融合机制

AI接收的是经过统一预处理的256×192RGB图像，分割模型输出目标所在的区域。后处理环节把掩膜转换成几何候选，再结合用户的点选位置、物理边界以及当前帧验证来确定最终目标。AI给出的候选还要经过当前帧验证，才能作为可以输出的校正结果。当多本书的掩膜连在一起、边界本身就很模糊，或者目标在跨帧之间发生移动时，仍然可能产生把几本书一起框住的大框，或者拿到过时的四角。所用模型为MobileNetV3-Small编码器加轻量U-Net解码器[2],[3]，共143层，权重5.46MB[2-3]。推理采用NCNN FP32，在ARMv7上单线程运行，实测单次耗时中位数约28.5ms[4]。模型进程常驻内存，只在需要时被调用，正常的校正输出不受影响。

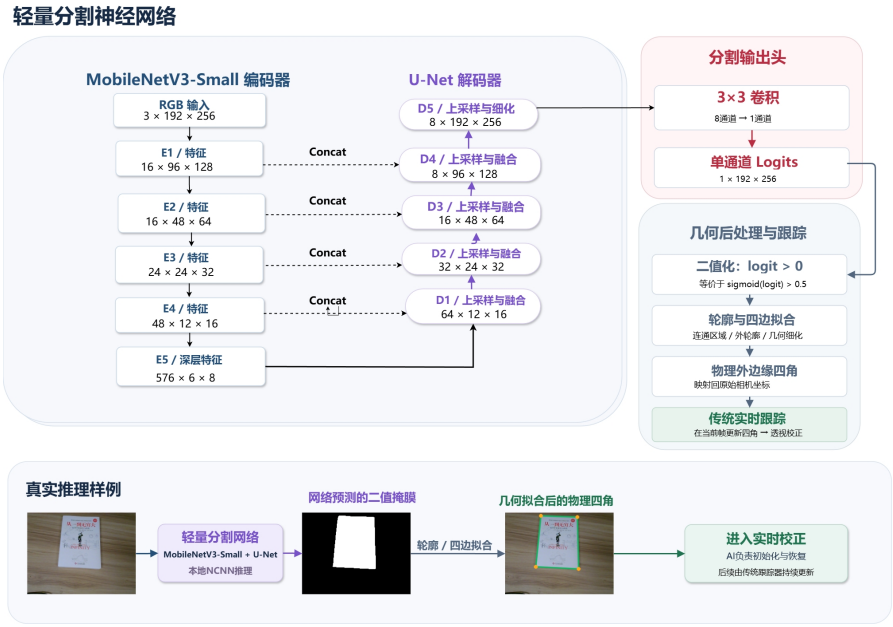


图4　轻量分割网络的完整结构与一次真实推理样例

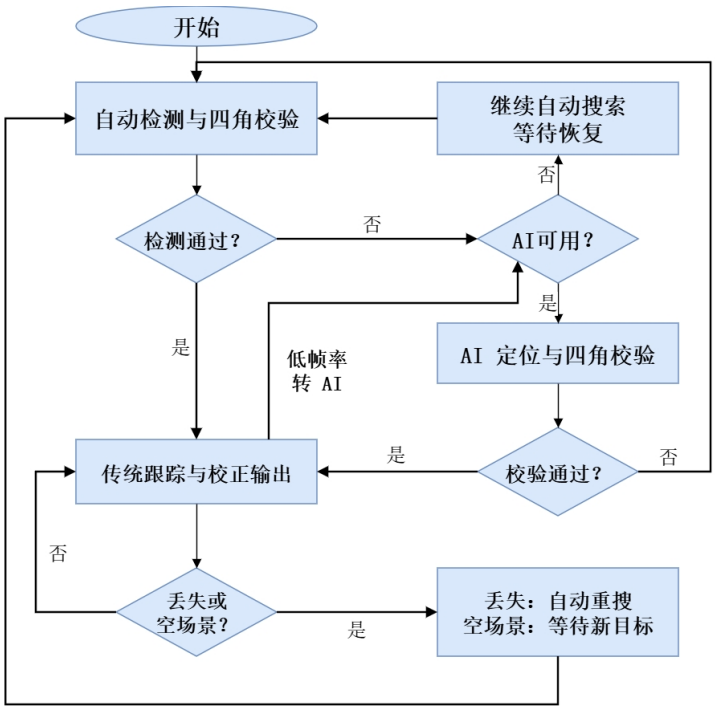


图5　自动检测、AI补充定位与传统跟踪的调度关系

传统方法与AI的融合体现在三个层面上。第一个层面是搜索调度，常规场景先交给传统方法处理，遇到困难场景才按需启用AI。第二个层面是候选验证，AI提供的是区域先验，具体边界还要由传统边缘和几何关系来约束。第三个层面是时间上的连续性，AI一旦找到目标，就把初始四角交给后续的跟踪模块接管，逐帧推理到此为止；之后是否需要恢复，由跟踪的质量来决定。

用户主动点击AI校正后，在还没有找到目标之前，系统会一直保持这次AI定位的意图；目标被找到并正常输出之后，即使出现短暂失效也先延续当前画面，直到满足原有的丢失条件才退回自动恢复流程。

多本书场景下的点选，需要在480P和600P两个档位里都统一把显示坐标换算成相机坐标，然后把选中的目标意图传给AI。需要注意的是，包围框包含点击点只能说明这一下点在了框内；要确认选中的是哪一本书，还需要把目标可靠地拆开。拆不开时，系统报"选择未确认"，让用户重新点选。

2.3.4 跟踪与恢复控制

跟踪模块的输入是当前相机图像、已经确认的四角以及同一目标的参考信息。系统沿着目标的物理边缘建立证据，再结合特征跟踪、前后向一致性和几何验证，判断目标运动之后给出的四角是否仍然可信。对于白书放在白底上或者纹理很弱的平板这

类情况，只靠纹理点并不足以获得可靠的约束，因此还要同时利用四条边的观测结果和目标内部的参考信息，防止跟踪结果吸附到背景纹理或者平板内屏上。

光流跟踪不逐帧运行，只在四种情况下触发：边缘证据不可靠、目标运动突然加剧、累积漂移超限，或锚点帧龄到期。触发时用当前帧与锚点帧做一次双向LK，求出的位移要通过下面的门控，才允许重新解算单应矩阵。

LK使用21×21的窗口。快速路径的金字塔最多3层、迭代12次；全路径最多4层、迭代20次，位移收敛阈值0.02像素。目标外缘上最多取100个特征点，每个点的前后向误差小于1.4像素才计入；同一个锚点上，前后向计算始终使用同一图像通道。

两条路径按运动幅度选择：小运动先走预测快速路径；锚点位移超过12像素、或采集步长不小于6帧时直接走全路径；只有在快速路径求解失败、且时间预算仍允许时，才升级为全路径。

门控由edge_gate判定，要求四条边齐全、最小支持率不低于0.60、最大拟合残差不大于1像素、修正量不超过搜索半径的0.65倍，并且四条边的极性一致。锚点本身有帧龄上限，最大间隔30帧；到期只做一次局部复验。静态保持期间还要受可信期限的约束。

有四种情况会拒绝更新：有效点少于4个、RANSAC内点率低于0.65、角点越出图像边界，或者当前帧的物理边缘复核不通过。被拒绝时不更新位置，直接交给恢复逻辑；局部恢复失败之后才扩大搜索范围，避免对同一个候选反复验证。

出现短暂失效时，系统最多延续1秒，用最后一次可信的四角去校正当前相机画面，左右两路仍然保持更新。这个计时从最后一次可靠确认开始算起，中间靠估计得到的帧不会反复延长可信期限。一旦确认目标已经离开、换成了另一本书，或者运动幅度大到无法可靠跟随，旧的位置就会被清除。确认丢失之后，系统采用1.2秒的确认条件和自动优先的恢复顺序。自动AI的失败请求仍然受预算约束，而在成功并稳定3秒之后，可以根据现有证据退还这次请求的名额。

2.3.5 平面透视校正

系统把有顺序的四个源角点与输出矩形的四个角建立对应关系，据此求出这个平面的投影映射[5]。PL侧采用逆向映射，为每一个输出像素反查它在输入图中的位置，这样可以避免正向映射时散射造成的空洞。选择"原比例"输出时，画布布局由目标本身的几何关系决定。由于四角一旦出错就会直接改变映射并造成画面拉伸，定位与跟

踪结果的可信程度，比在显示端再加一层滤镜更为关键。

2.3.6 增强参数、帧率与并发管理

增强参数按最新值优先，最多每100毫秒更新一次，并在下一帧开始前统一生效；调整过程中相机保持运行，角点与定位模式沿用当前状态。

帧率只统计新的有效校正帧成功提交到HDMI之后的情况，重复重绘、冻结画面和仅显示原图都不计入。交付版本取最近3秒的计数除以3，每100毫秒刷新一次。这个指标反映的是有效输出吞吐，与面板的物理刷新率、单帧耗时的倒数都是不同的量。

并发方面，AI服务由独立线程负责加载、推理与退出，结果带代次与帧回执，过期结果直接丢弃；相机复用省去反复打开设备的开销，但改变分辨率仍要重配视频流并重新定位；系统始终只保留一个HDMI提交者，队列长度有限，避免多路显示争用与任务积压。

第三部分　完成情况及性能参数

3.1 整体介绍与工程成果

本系统基于PYNQ-Z2平台实现，PS端完成相机采集、目标检测、四角跟踪与单应矩阵规划，PL端实现光补、锐化与重映射校正的硬件流水线，校正画面经HDMI输出。系统实现了传统自动校正与AI校正的协同，同时包含手动校正功能。自动校正通过边缘检测与光流跟踪实时估计目标四角，本地神经网络会在传统校正过慢或失败时接管校正。校正过程中实现了运动中实时校正、放大阅读、画面冻结与截图等功能；图像处理中实现了输入画质切换（480P/600P）、光补、锐化。为改善人机交互体验，系统采用全触摸屏操作界面，全部操作均可通过多点触控完成。

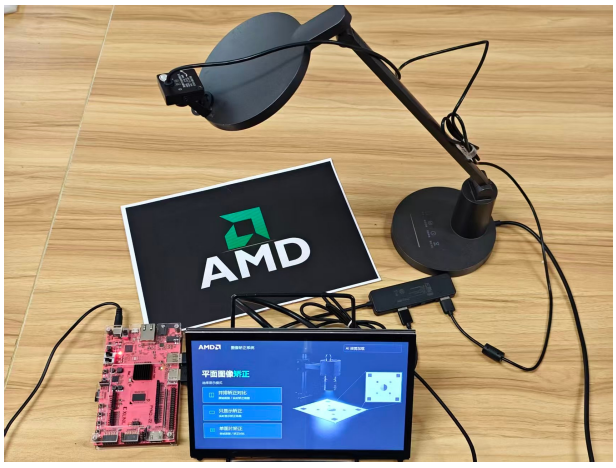


图6　完整作品：PYNQ-Z2板卡、7寸触摸一体屏、台灯臂夹持的USB摄像头

软件层面，完成MobileNetV3-Small + U-Net轻量分割模型训练与NCNN FP32板端部署，模型约137万参数。构建传统检测优先、AI辅助校正、统一跟踪更新的协同机制：传统检测超时或失败时调用AI，分割结果经几何拟合与当前帧边界校验后交由跟踪器持续更新，避免逐帧运行网络推理。同时实现目标点选、丢失恢复、模式调度、运行日志与性能统计。



图7　系统首页

3.2 特性成果

3.2.1 传统自动校正

自动校正的效果如下图，并排校正帧率可达19帧/秒。



图8　并排自动校正对比实拍

3.2.2 手动校正

当用户对系统自适应校正的结果不满意时，可随时进入手动校正模式。用户在触摸屏上依次点选目标的四个物理外角，支持确认、撤销、重标与放大辅助选点，确认后立即计算单应矩阵并经PL完成校正输出，与自动校正共用同一校正流水线。

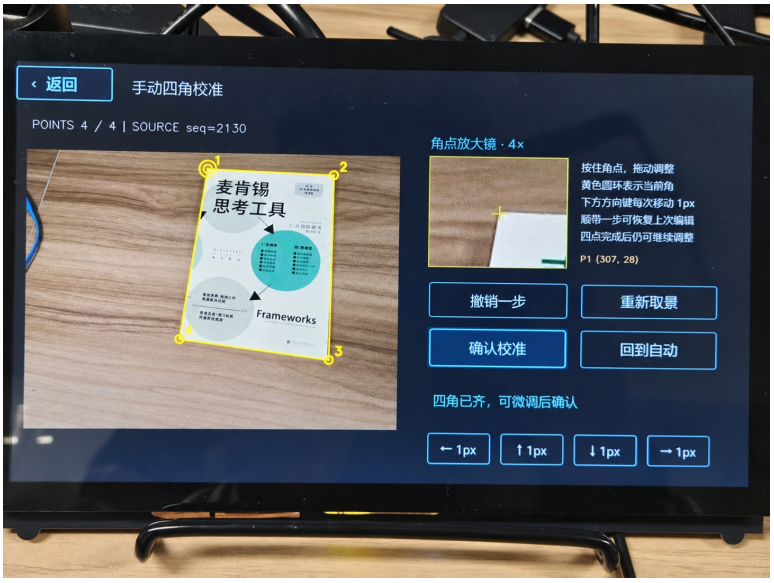


图9　手动校正界面

3.2.3 放大阅读

系统提供放大阅读功能，用于细读校正画面中的小字。放大窗口支持两种拖动方式：在左侧总览图上直接拖动，焦点随手指移动；在放大窗口内拖动，则按当前倍率平移观察范围，实现拖动式阅读。放大支持2倍和4倍。

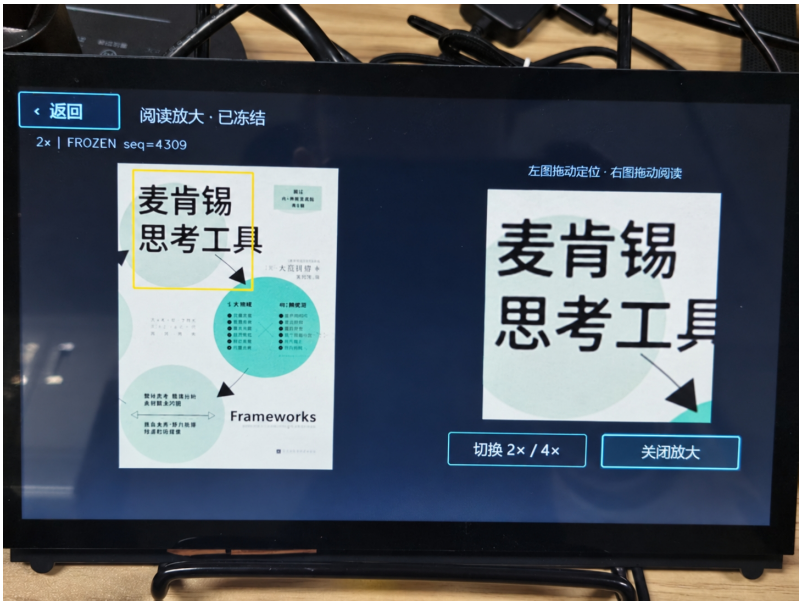


图10　放大阅读演示

3.2.4 显示比例切换

系统提供显示比例切换功能，用于按书页、平板等阅读对象的实际形状调整校正输出画面的比例。点击界面右上角「比例」按钮打开右侧设置面板，面板提供「原始、1:1、3:4、4:3、16:9」五种预设及「自定义」比例。



图11　显示比例切换

3.2.5 多目标切换

当画面中存在书页、平板等多个平面目标时，系统支持通过点击直接切换校正目标。若点击位置仍落在当前目标四角内，则判断为重复点击并保持当前跟踪，不触发切换。



图12　多目标切换

3.2.6 校正图像处理

系统在PL校正流水线中集成了光补与锐化两种图像增强功能，用于改善拍摄光照不足或画面偏软时的阅读效果。光补采用PL端分段线性（PWL）暗部提升算法，强度可在0～100%之间调节，提供0、20、40、60、80、100六个预设档位及连续滑条。锐化采用PL端流式锐化内核，强度可在0～4.0之间调节（共40级），提供0、1.0、1.5、2.0、3.0、4.0六个预设及连续滑条，用于增强文字边缘的清晰度。



图13　图像处理

3.2.7 相机分辨率调节

系统提供480P与600P两档相机输入分辨率切换：480P为640×480@30fps，600P为800×600@15fps。



图14　相机分辨率调节

3.3 训练数据与模型验证

模型沿用此前训练完成的结果。数据包括历史样本合计469张，其中训练集379张、验证集36张、测试集54张。训练共80轮，选定最佳轮次后冻结参数，再在测试集上评估。在训练组里，它们评估时的改善说明的是模型对这些样本的拟合程度，独立泛化要看测试集上的数字。

表4　模型在历史验证集与各数据组上的表现

数据范围	旧模型通过	新模型通过	平均角点误差
历史验证集	36/36	36/36	3.90 → 3.67px
保留测试集	53/54	53/54	4.63 → 4.29px
训练样本1	14/27	27/27	123.31 → 1.91px
训练样本2	34/36	36/36	6.55 → 1.66px

定位的通过条件是：在640×480坐标系下，四角平均误差不超过10px，单角最大误差不超过20px。评估器会把未检出的样本按999px计入均值，所以报告里看到的误差大幅下降，有一部分来自漏检变少，不全是定位变准。[8]

3.4 实时运行与验证边界

板端的运行记录来自同一份run-result.log，共56,606条。这份日志只覆盖开启记录功能的时段；拍摄演示与部分现场测试为了降低开销没有开日志，所以下面的数字是下限，实际运行时长和交互次数都高于此。其中有记录的运行累计4.8小时、18次程序启动、232次交互会话，共处理79,073帧，其中65,673帧为有效校正输出；其余帧大多来自画面里没有目标、或正在切换目标的过程。崩溃捕获机制在每次启动时均已装载，运行期间没有捕获到任何信号。有记录的最长一段无中断连续运行约49分钟。

同一份日志里还能看到早期版本的一次板端运行：画面先用600P再切到480P，并切换过双画面与单校正两种显示方式。[7]

表5　早期版本与交付版本的输出帧率对照

场景	输出FPS采样范围	说明
早期版本600P双画面	11～19	包含两次双画面显示阶段
早期版本480P双画面	16～21	剔除刚获得输出的窗口建立点
早期版本480P仅校正	16～27	21个采样点，中位数25
交付版本实测480P并排	6.5～19（中位8.0）	285个采样；p90 14.5，最大18.0
交付版本实测480P仅校正	18～27（中位17.8）	156个采样；p90 25.5，最大26.5

表格中前四行来自早期版本日志，只作历史对照；后两行是对交付版本按相机序号逐帧重新统计的结果，口径为“有效输出帧数除以实际耗时”。

并排模式与单校正模式的帧率差距来自显示合成这一环。并排模式每帧要同时合成原图与校正图两路并叠加界面，实测合成耗时中位数39.4ms、均值45.2ms。单校正模式只合成一路，中位数20.6ms、均值23.5ms，两者相差约20ms，这正是两种模式帧率差的主要来源。此外，计算线程与显示线程同处一个Python解释器，共享同一把全局解释器锁。当计算侧持续占满时，显示合成被迫让出执行机会，实测出现过单帧处理时间变短而并排帧率反而下降的情况。

静态段与动态段的帧率差别，来自跟踪是否触发复测。目标静止时系统复用上一步的几何结果，单帧处理中位35.3～36.6毫秒；目标运动时要逐帧做光流复测，复测帧的整帧中位升到142.5毫秒，其中LK本身占71.6毫秒。所以静态段帧率明显高于动态段，动态段又会随目标运动幅度波动，表1里的平均帧率是这两段合起来的数。

早期还做过一版取舍：把动态帧率拉平，不让它随运动下降。代价是动态段的画面跟随明显变差，目标一旦移动，四角更新滞后，校正画面出现拖影。两版对比之后选了现在这一版——动态段优先保证跟得住，静态段利用保持把帧率拉高。

光流跟踪在这套门控下是稳定的。复测只在必要时触发，触发之后只有两种结果：通过门控就更新位置，不通过就判失败交给恢复，不会停在中间状态；没有触发复测的帧由四边证据直接确认，不额外消耗算力。

3.5 PL 校正核的设计迭代与验证

整个开发过程中，校正核的吞吐累积进行了四轮迭代，提升至每像素4.686个时钟周期，累计提升2.56倍。

表6　校正核吞吐的四轮迭代

轮次	主要改动	时钟周期/像素	相对优化前
优化前	串行状态机，取坐标、读邻域、插值依次执行	11.99	1.00倍
第一轮	坐标流水级独立成级，三级真正并行	7.160	1.67倍
第二轮	坐标发生器除法并行度8 → 16	6.675	1.80倍
第三轮	行缓存按列地址低位分两体，读邻域5 → 3个时钟周期	5.679	2.11倍
第四轮	插值级状态5 → 3，去掉本级的邻域锁存	4.686	2.56倍

第一轮把负责坐标的流水级也拆成独立的级，使坐标生成、邻域读取和插值三级能够并行处理不同的像素，每像素从11.99个时钟周期降到7.160个。

第二轮把坐标发生器的除法并行度从8提到16，除法环节的占用从6.125个时钟周期降到3.06个，整核随之降到6.675个时钟周期。

第三轮把行缓存按列地址低位分成两个体，使左右两列的读取可以并行，同时压缩了读邻域那一级的状态数，整核降到5.679个时钟周期。

第四轮把插值级的状态机从五个状态压缩到三个，去掉了这一级自己的邻域锁存，在同一个时钟周期内直接送给插值单元，最终降到4.686个时钟周期。

除了吞吐，还有一轮是为时序收敛做的。插值单元最初把横向与纵向插值合在一个时钟周期内完成，综合报告给出的最差路径是16.9ns，在10ns的周期下严重违例，WNS为−6.878ns。拆成两级流水之后收敛到+0.488ns。

第一轮之后的瓶颈判断曾经出过错。当时看到“读邻域那一级等待上流的时间占了5.2个时钟周期”，便认定它是瓶颈，于是把行缓存分成四个体、把这一级从五个时钟周期压到两个。十七个测试平台全部逐字节通过，功能完全正确；但吞吐只从7.160变成7.162，几乎没有变化，而BRAM却从96块涨到了128块，占整机的91%，最终布不通。回头再看才发现，瓶颈并没有消失，只是从“等前一级”搬到了“等后一级”：这一级变快之后，后面插值级的等待时间立刻涨了回去。这个教训后来固化成一条规则：改动之前先列出所有并行级各自的每像素占用，取其中的最大值，并预先算出改完之后这个最大值会变成多少；如果最大值不变，这次改动就不会带来任何吞吐收益。

此外，还尝试过两个已被证伪的参数：一是把除法并行度取成12，算术上够用也省资源，但非二的整数次幂会让仿真出问题。二是把行缓存深度加到448行，可以让两

分块变成一分块，综合探针却显示需要128块BRAM，占整机94.3%，同样布线出现问题。这两个参数最终都退回到二的整数次幂与已验证的384行。

功能正确性的验证采用了双参考判据。每一级RTL都有一份独立的定点行为模型，测试平台同时比对两份参考，任何一方的差异超过零都会被判定失败。目前共十七个测试平台参与回归，覆盖单块与多块、横向与纵向、边界钳位与饱和输出等场景，逐字节一致的结论是每次改动的放行条件。这套判据在四轮迭代里一共抓出三个只有在特定几何姿态下才会触发的缺陷，其中两个是“综合能过、仿真不过”的类型，靠上板试是试不出来的。

表7　优化前后的性能与资源对比

项目	起点（彩色首版）	校正核优化	交付版
校正核吞吐（时钟周期/像素）	11.99	7.160	4.686
Block RAM	96 / 140（68.6%）	100 / 140（71.4%）	121 / 140（86.43%）
LUT	8,539	15,959（30.0%）	17,569（33.02%）
DSP	15 → 29	40（18.2%）	46（20.91%）
WNS	−6.878ns	+0.488ns	+0.650ns

第四部分　大模型协作与技能包

4.1 智能体工作流

因FPGA开发具有接口约束多、验证周期长，资源与时序相互制约等特点，其工作流相应分成三层：项目约束层、任务执行层、验证反馈层。约束层保存设计要求，执行层组织工具操作，反馈层判断任务结果；代码修改始终受接口约定与验收条件的约束。

项目约束层保存跨任务共享的设计要求：帧率优先的性能目标、目标物理外边缘定义、H 估计降级链、模块接口、已验证参数。长期约束写入项目指令文件；每轮任务开始前，再明确本轮修改目标、允许修改的模块和预期输出。长期要求与当轮任务分开管理，上下文变化时接口保持一致，已经排除的方案不再重复尝试。

任务执行层按“读取相关上下文、分析问题、制定修改步骤、实施修改、调用工具”的顺序工作。智能体根据任务读取相关代码、测试用例和历史报告，在限定范围

内辅助修改，通过工具接口执行 Tcl、仿真、综合实现及数据统计。任务以能独立检查的修改为单位拆分，结果变化能对应到具体改动，失败后也容易定位和回退。

验证反馈层分阶段验收。先做 RTL 仿真，与定点参考模型比对，检查功能正确性；功能回归通过后，再看综合实现结果：平均像素处理周期、资源占用、关键告警、WNS。仿真失败，回到差异分析和代码修正；功能正确但性能没改善，或资源、时序不满足要求，则重新评估方案，由团队决定继续调整还是回退。阶段是否完成，以验收结果为准。

工具链检查通过的版本进入板端验证：团队检查实际输出、帧率、交互，以及不同场景下的稳定性。板端发现的问题重新整理成带触发条件和预期行为的任务，进入下一轮闭环。每轮保留修改内容、验证结果、关键指标和方案取舍记录；确认有效的约束和参数写回项目上下文，后续任务直接复用。

4.2 提示词与自我纠错

我们将提示词按用途分成几类，每一类都有固定写法。通用的一类是让它动手：写清范围和工具，才能直接拿到可运行代码。第二类是授权自主执行，交代一次就不用逐步确认。第三类是只要诊断、不要修改，必须写明不要改任何代码，否则它会顺手改掉，对照基线就没了。第四类是先复现再修，把先复现、后修改的顺序写死。按任务类型另有五类专项模板：软硬件划分、按综合报告定位瓶颈、先把判据做红、让它先报口径、再报数、先证明这条路径确实会跑到。

比模板更要紧的是把约束写进文件。写在对话里的要求会随上下文滚出窗口，写进项目指令文件的每一次都会被读到。固定下来的是四类东西：硬约束，说明哪些文件不能动、哪些配置已经验证过；接口契约，寄存器表、AXI 位宽与时钟频率；验证判据，即怎么算通过；以及交付与打包规则。这里有一条可操作的判据：同一个要求如果在对话里重复到第三次，就该把它落到文件里。这条规则之后，同类约束没有再一次遗漏。

指令文件分两层。全局那一层跨项目通用，写的是怎么协作：回答要短、先给可操作的步骤，不铺陈；文件优先放在数据盘，不往系统盘写大文件，临时文件用完即删；工具调用不逐条弹权限打断进度。还有一条记录同步铁律：方案或决策变了，要改对应提示词的那一节；代码或配置改了，要改受影响的提示词和引用它的文档；实测数据变了，要改所有引用这个数字的文档，如果它改变了结论，还要回头改结论所

在的那一节；每篇记录末尾附一张同步清单，逐条标注已经表态还是待办。此外每条回复以协作记录摘要收尾，写明问题、结论、待办与踩坑；每个任务先给时间盒，超出预算主动给降级方案。项目那一层写这个工程自身的事实：工具链的路径与版本、哪两套调试接口不能混用、哪些配置已经验证过、交付与打包规则、文档放在哪里。两层配合下来，新开一次对话就能直接接着做，规则在对话变长之后依然生效。

效率提升最大的是授权自主执行，即告诉它过程里的操作自己执行，一次交代即可。但这条有个前提：项目里必须已经有能自动判定通过与失败的东西，比如测试平台的逐字节比对、检查脚本里的断言。没有判据就授权，等于让它靠猜。配上逐字节判据之后，一次授权之内它就能走完改 RTL、跑仿真、综合、实现、出比特流的全过程。

让它自己找问题有三个模式：一是让它构造必然触发的用例，要构造出来，它必须先把触发条件想清楚。二是撤掉修复、跑同一个用例看症状能否复现，这一步把改完测试通过升级成修前修后症状对得上，三次应用下来症状全部吻合。三是给它现场数据：先让程序把状态寄存器、分段耗时、出错行号打出来再交给它，从数字推因果比从现象里猜准得多。问一句为什么这么慢，它猜的是五秒，实测是二十六秒。

专项模板的价值在判断题上体现得最直接。动手优化之前先问一句软硬件划分，实测下来 PL 只占整帧的百分之一，据此决定不去优化它。省下的是不必要的工作。判断该往哪儿使劲时，把综合报告整段交给它，一次问答就排除了加宽、提频、压缩时序三个方向，避免了一轮为期一周的重做。要求它先证明某条路径确实会执行到，也查出一个默认关闭、从未打开过的开关，打开后每帧省下 76.75 毫秒。

自我纠错的底子是实测反馈与代码分析相互印证。PL 输出整体错位一格时，先验握手时序、再验行缓存深度，两次改动都没有变化；回去核查实际参与综合的源码，才发现改的是一份没有被引用的副本。实机现象、构建记录和代码必须对得上，模型给出的假设也要逐条验证。这类复盘固化成了先验证前提、再继续推理的协作规则，并写进项目指令。开发记录保留了九个纠错案例，包括性能估算偏差靠分段计时纠正、陈旧状态位一度带偏排查方向、以及连续两个假设被证伪，供后续方案审查时对照。还有三件事始终不交给模型：删除或覆盖文件、对外发布、改已经验证过的配置。前两件一旦发生就不好回退；第三件常以顺手的小改动出现，破坏的却是原本正确的行为。

4.3 技能包的提炼过程与边界

团队把项目中有效的方法归纳为提示词工作流、案例模板、校验脚本和踩坑清单四类，大模型协助整理结构、补齐说明、统一格式。

条目都从实际调试里提炼：团队给出故障现象与运行条件，配合模型完成复现、定位和修复，再补回归用例。流式握手的条目对应输出错位、下半图拖影和DMA挂起三次上板故障；行缓存深度的条目来自估算与实测相差三十倍的那次复盘，两者都保留了具体证据。

有效性用两种方式检查：撤掉修复看问题能否重现，以及从零跑一遍完整构建；最后再结合板端效果复核。技能包沉淀的是团队与智能体共同验证过的方法，以及它们的适用边界。

边界也写明：不含完整算法实现，不含HLS，不含高速接口，不含多板卡经验；全部经验来自Xilinx Zynq-7000与Vivado2025.2，换大版本后部分报错信息会变。

4.4 AMD ROSS 智能体套件的使用心得与优化建议

对AMD ROSS套件，我们用了两种方式验证它的功能性：一是自2026年10月2日起把它接入本项目的日常工具链，在真实开发里边用边看；二是在最终交付版本敲定之后，用一次“盲重建”专门试它的独立作业能力。

日常开发中的便利。一是会话常驻，Vivado的进程在对话结束后不会退出，连续调试时不必反复冷启动，这一条省得最多。二是长命令返回运行中状态后自动交接，我们可以并行做别的事。三是没有图形界面也能拿到全部现场，告警原文、端口清单与IP锁定状态都直接返回，排查时不必截图对照。可复现的量化记录有两条：启动Vivado并跑通第一条Tcl用时14.2秒，手工开界面约需3到5分钟；打开已交付工程读取实现报告用时77秒。

工具行为上还有四处不便。需要说明的是，它省的是敲命令和等界面的时间，设计决策与参数取舍仍由人完成。四处不便分别是：低版本打开高版本工程时返回只读，要自己想到先另存为低版本；工程引用的IP仓库缺失时，报错要到综合阶段才暴露，而且表现为顶层被自动替换成别的模块，排查成本很高；长命令只有运行中与成功两种状态、没有进度，而综合实现动辄几十分钟；重复启动构建时不检查是否已有构建在跑，会和自己撞上。

除此之外，判断行缓存要多深才够时，我们用的是create_project -in_memory加synth_design -mode out_of_context，在一个会话里连跑九个配置，把“BRAM够不够”从推算变成了实测。这类只针对单个模块的问题，走OOC比跑全设计快一个数量级，结论也不受其他模块牵制。

盲重建的做法。把本项目的PL源文件、IP仓库与约束连同任务书一并交给它，全程禁止读取原工程。为确认“盲”是真的，团队事后逐条复核了它全部的读文件记录，553条路径无一落在原工程上。

盲重建凸显的优势。它独立跑通了综合、实现与写比特流，产出一份389行重建报告，含各时钟的WNS与全设计文件的哈希表。产物上板验证：加载15.16秒，13个IP全部识别，读AXI DMA的寄存器能返回正确值（0x00011003）；主时钟的WNS为+0.248ns（83.333MHz），另一时钟域为+0.643ns，DRC无报错。过程一共跑了六轮，全程没有人插手：前三轮是多个子模块的独立综合任务并发时互相冲突，它把并行任务数由8改为1后通过；第四轮一个子任务的综合进程崩溃，它只重跑那一个、其余成果保留；第五轮布线阶段静默退出，无报错也无崩溃记录，它靠缺失的阶段标记判断这一步没跑完，复位后重跑。

盲重建暴露的短板。它的验证只覆盖“设计能不能造出来”：文件生成、时序收敛、DRC无报错，这三条讲的是工程完成度，回答不了“造出来能不能用”。整轮重建里没有把一帧图像送进流水线，也没有核对输出，比特流在功能上仍未经检验。另一次试验把这一点暴露得更彻底：让它从零写一个透视校正IP，综合、实现、比特流都能写出，仿真也通过，但上板后4KB地址空间里1024个寄存器全部为0：总线握手能完成，主控不会挂死，但读数据不被寄存器表驱动、未生成写使能。它的仿真覆盖了三个算法核，恰恰没覆盖寄存器的读写时序，因为没有人要求它覆盖。

补齐判据之后，输出的可用性明显提高。第一轮把验收条件改成“每个寄存器写进去要原样读回”，它二十分钟就定位到读数据寄存器被两处逻辑同时驱动，这个缺陷仿真看不见；修好后4KB地址空间的非零项由0变为896。第二轮要求“一帧要传得动”，它查出DMA单次传输的长度位宽沿用了默认值，单次上限只有16,383字节而一帧需要1,228,800字节，改成更大位宽后重建；同轮它自己报出下一个阻断点：流水线每输出一行就发一个行结束标志，而DMA的简单模式会把任何一个这样的标志当成本帧结束，并且不报错。

第三轮据此修改了三处帧结束标志的生成逻辑，其中一处不在任务书里、位于旁路分支，是它自己找出来的；配套的静态检查对旧代码报3处失败、对新代码报0处。第三轮的比特流因本机综合工具环境故障未能重建，它没有因此写“已完成”的报告。

结论。两次验证放在一起，可以说得更准确：ROSS的“跑对”能力取决于有没有一条能判红的判据。给弱判据，它把活干完但功能一个没验；给强判据，它二十分钟就能挖到只有综合语义才暴露的缺陷，并顺手把自己的验收标尺也改对。所以本项目的用法是：人负责定义“什么叫对”，它负责把“对”做出来。

优化建议。七条均来自实际踩到的点：

表 8　AMD ROSS 套件的七条优化建议

#	问题	建议
1	低版本打开高版本工程时返回只读	打开阶段就提示可另存为低版本
2	IP 仓库缺失要到综合才暴露，且表现为顶层被自动替换成别的模块	打开工程时即检查 IP 仓库是否齐全
3	长命令只有运行中与成功两态、没有进度	显示 Vivado 自身的进度百分比
4	交付前没有功能门禁	比特流出包前必须有寄存器读写回环与端到端数据通路（送一帧进去、核对输出）两类证据，缺了不许写通过
5	生成的寄存器从设备没有协议级测试	自动附一份测试：遍历地址读默认值、写图案再读回、检查应答信号是否真的拉起来
6	通过标准混用，“全部通过”会被读成“东西成了”	拆成时序、加载、功能三档；“未做功能验证”这类限制写在开头
7	重复启动构建时不检查是否已有构建在跑	启动前检查进程或状态文件，加一道互斥

第五部分　复现说明

复现分两条路径，按要做的事选一条。环境统一为PYNQ-Z2（xc7z020clg400-1）、板上PYNQ v3.1镜像（pynq3.1.1 / Python3.10.4），EDA用Vivado与Vitis2025.2；交付的位流就是在这套环境下由仓库脚本建出来的，按路径B可重建出同一设计。

路径A直接上板，约15分钟。上板对象是整个发布目录，共115个文件、26.5MB[6]，必须整个目录一起传——启动时会按清单逐个核对顶层94个文件的哈希，缺一个或内容不一致就拒绝启动。上传脚本先把文件写到临时目录，逐文件核对后再就地覆盖，不一致则中止并保留旧目录。硬件要求四项：PYNQ-Z2板卡、接在HDMI OUT的显示器（输出固定1280×720）、固定为/dev/video0的UVC摄像头，以及支持绝对坐标的USB触摸屏；缺触摸屏时启动会直接报错。板上打开Jupyter，进入该目录运行

启动Notebook，先重启内核再依次执行前两个单元；运行时点一下触摸屏开始处理，界面显示帧率读数，默认运行时长上限两小时。

不开相机也能先验一套：运行AI自检Notebook，用包内10条用例图跑一遍分割，全部通过即说明AI子进程、清单校验与图片素材都完整。

路径B从零重建，约1小时，另加综合与实现的时间。PL侧用Tcl脚本从Block Design源、7个已打包的IP与RTL建工程，再依次跑综合、实现、写位流，导出9份报告与硬件描述文件；放行条件是WNS大于0、DRC错误为0、布线错误为0。PS侧源码为纯Python，直接运行；6个C++文件在板上首次运行时由g++现场编译成动态库。最后用打包脚本合并成发布目录，经过三道校验才能出包。自己重建的位流要上板，需先改齐各处的哈希登记并逐条回读核对。

两条路径的完整命令、目录结构与逐层验证方法，见开源仓库的README。

第六部分　总结

6.1 可扩展之处

已交付版本在三处仍有边界。多个同色目标贴在一起时，系统能发现掩膜相连，但还不能自动拆成独立目标，只能报“选择未确认”让用户重新点选；AI服务是单工作线程加命令队列，同一时刻只处理一个请求，目标快速切换时会排队；白书放在白底、平板边缘与桌面颜色接近时，低对比重试能救回一部分，纹理极弱时仍会丢跟踪。

更远的方向分两类。一是换相机与接口：现在走USB2.0，可用带宽约35MB/s，640×480@30fps已占去18.4MB/s，改用USB3.0或MIPI之后分辨率与帧率可以一并提高——校正核有17.8兆像素每秒的处理能力，相对相机9.2兆像素每秒还有近两倍余量，这条路对PL的改动很小。二是在现有链路上做增强，例如用亮度网格估计缓变光场做局部光补、或用网格形变处理弯曲书页；这一类要额外占PL资源，而BRAM已用到86.4%，得先腾出余量。

6.2 设计总结

回看整个过程，收益最大的做法是“先量再改”。每次动校正核之前，先把各并行级各自的每像素占用摊开，取其中的最大值，并预先算出改完之后这个最大值是多少；如果最大值不变，这次改动就不会带来吞吐收益。这条规则是被一次失败教出来

的：当时把行缓存拆成四个体，十七个测试平台全部通过，功能完全正确，吞吐却只从7.160变成7.162，而BRAM从96块涨到128块、占整机91%，最终布不通。回头再看，瓶颈并没有消失，只是从“等前一级”搬到了“等后一级”。

保证正确性靠的是双参考判据。每一级RTL都配一份独立的定点行为模型，测试平台同时比对两份参考，任何一方差异超过零即判失败；十七个测试平台覆盖单块与多块、横向与纵向、边界钳位与饱和输出。四轮迭代里，这套判据一共抓出三个只在特定几何姿态下才触发的缺陷，其中两个属于“综合能过、仿真不过”，靠上板试是试不出来的。

AI与传统算法的分工是按代价定的。AI只负责初始化和恢复，给出的候选先过当前帧验证，再交给跟踪模块；网络只在初始化和恢复时介入，省掉了持续的推理开销。代价是首次进入AI路径要等一次推理，实测候选返回约2.49秒，明显慢于传统路径的0.38秒，所以它只在常规搜索失效时启用。

标注口径的统一看起来是数据工作，实际决定了整条链路能不能对上。所有样本都取目标的物理外边缘，平板把黑色边框算在内；这样模型输出的区域、跟踪器维护的边界、以及最终用于单应解算的四个角点，指的是同一个东西。如果模型学的是内屏、跟踪跟的是边框，两者在跨帧时就会互相拉扯。

评估口径同样需要分开。定位精度、显示读数和系统稳定性是三条独立的线：扩大帧率统计窗口只改善读数的响应速度，不改变处理速度；训练集上的表现只说明拟合程度，不代表陌生场景；功能能跑起来也不等于长时间运行稳定。报告因此把这三类结论分开陈述，各自注明适用范围。

还有一条是从跨芯片流水里长出来的。PS与PL在同一时刻处理的是不同的帧，一旦帧身份对不上，PL输出的就是过期画面。系统给每一帧带上帧序号与页面代次，异步返回的结果也要回执校验，旧任务的输出直接丢弃。这看起来是工程细节，实际是把流水做长之后必须补上的代价——级数越多，越要能回答这一帧是哪一帧。

第七部分　参考文献

[1] Bradski G. The OpenCV Library[J]. Dr. Dobb's Journal of Software Tools, 2000.

[2] Howard A, Sandler M, Chu G, et al. Searching for MobileNetV3[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). Seoul: IEEE, 2019: 1314-1324.

[3] Ronneberger O, Fischer P, Brox T. U-Net: Convolutional Networks for Biomedical Image Segmentation[C]//Medical Image Computing and Computer-Assisted Intervention (MICCAI 2015). LNCS 9351. Cham: Springer, 2015: 234-241.

[4] Tencent. ncnn: high-performance neural network inference framework[CP/OL]. https://github.com/Tencent/ncnn.

[5] Hartley R, Zisserman A. Multiple View Geometry in Computer Vision[M]. 2nd ed. Cambridge: Cambridge University Press, 2004.

[6] 交付版本软件与硬件清单[CP]. 2026-11-01.

[7] 模型训练与评估记录[Z]. 2026-11-01.

[8] run-result.log 运行记录[Z]. 2026-11-01.